Программная инженерия

Лекция 2: Культура командной разработки

Денисов Артём Владимирович

Уральский Федеральный Университет

2026

Роли в продуктовой команде

- Разработчики (Software Engineer / Dev)
- Tech Lead / Архитектор
- QA Engineer (Инженер по качеству)
- DevOps Engineer
- UX/UI Designer
- Product Owner / Product Manager

Полный цикл разработки

Команда умеет сама планировать работу, проектировать, разрабатывать, тестировать и выкатывать фичи от идеи до продакшена.

Разработчик (Dev)

Основной исполнитель в команде

Что делает?

- Превращает требования в рабочую программу
- Пишет код, обновляет автотесты и документацию
- Создаёт pull request, участвует в code review

С кем взаимодействует?

- С Tech Lead — по архитектурным решениям, ревью
- С Тестировщиками — по приёмочным критериям, сценариям тестирования
- С Product Owner — по бизнес-требованиям

Tech Lead / Архитектор

Отвечает за техническое направление продукта

Что делает?

- Проектирует и эволюционирует архитектуру системы
- Проводит ревью ключевых PR, задаёт инженерные стандарты
- Принимает важные технические решения, оценивает риски

С кем взаимодействует?

- С Dev — помогает с дизайном решений
- С Product Owner — согласует технические ограничения и сроки
- С DevOps — по инфраструктуре, требованиям надёжности

QA Engineer (Инженер по качеству)

Отвечает за качество и тестирование

Что делает?

- Прорабатывает тест-сценарии
- Создаёт и поддерживает автотесты (unit, integration, E2E)
- Проводит ручное тестирование сложных сценариев и регрессии

С кем взаимодействует?

- С Dev — по воспроизведению дефектов
- С Tech Lead — по тестовой стратегии
- С DevOps — по тестовым окружениям, внедрению тестов в CI/CD

DevOps Engineer

Настраивает автоматизацию, инфраструктуру и надёжность

Что делает?

- Настраивает и развивает CI/CD-пайплайны
- Управляет окружениями: dev, staging, production
- Настраивает мониторинг, алерты, логирование; реагирует на инциденты
- Участвует в релизах, откатах

С кем взаимодействует?

- С Tech Lead — по архитектуре, масштабированию

UX/UI Designer

Отвечает за пользовательский опыт

Что делает?

- Составляет Use case и User story, User flow
- Создаёт прототипы и макеты
- Участвует в ревью реализации, даёт правки по UX/UI

С кем взаимодействует?

- С Product Owner — по user story
- С Dev — по адаптивности, техническим ограничениям
- С QA — по тестированию визуальной части

Product Owner / Product Manager

Отвечает за ценность того, что делает команда

Что делает?

- Ведёт и приоритизирует продуктовый бэклог
- Участвует в планировании спринтов, общается с командой
- Анализирует метрики продукта, общается с пользователями и стейкхолдерами

С кем взаимодействует?

- С командой разработки — на встречах, созвонах
- С заказчиками проекта и конечными пользователями

Пример: Новая фича «логин пользователя»

1. **Project owner** формулирует задачу и критерии успеха
2. **UX/UI-дизайнер** проектирует флоу и макеты логина
3. На планировании **команда** дробит задачу и оценивает объём
4. **Dev** реализует API и/или UI, пишет автотесты, открывает PR
5. **Tech Lead** проверяет архитектуру и качество реализации в PR
6. **QA** разрабатывает тесты и проверяет фичу на разных сценариях
7. **DevOps** обеспечивает готовность окружения, проводит деплой
8. **Project owner** принимает фичу по критериям приёмки, фича уходит к пользователям

Что идёт не так?

Разный стиль кода. Каждый пишет по-своему

- Команда большая
- У каждого свой стиль кода и свои привычки
- Команде трудно читать и сопровождать код

К чему это ведёт

- Потеря целостности проекта
- Рост стоимости изменений
- Меньше внимания архитектуре
- ...

Что делать?

Способы решения

- **Документы в репозитории**:
  - README.md
  - CONTRIBUTING.md
  - CODE_STYLE.md
- **Инструменты**:
  - Линтеры, форматтеры
  - Проверки в CI пайплайне

README.md

Содержание

- Что за проект и зачем он нужен
- Как запустить локально, какие зависимости установить
- Ссылки на подробные правила и документацию, др. документы

Пример

`https://github.com/immich-app/immich/blob/main/README.md`

CONTRIBUTING.md

Содержание

- Как правильно вносить изменения
- Ветки: как называть, откуда ветвиться
- Перед PR: какие команды прогнать (линтеры, тесты)

Пример

`https://github.com/immich-app/immich/blob/main/CONTRIBUTING.md`

CODE_STYLE.md

Содержание

- Конкретные правила код-стайла проекта
- Форматирование, именование, структура файлов
- Примеры «плохого» и «хорошего» кода
- Объём: 1–2 страницы, только важное

Пример

`https://github.com/google/filament/blob/main/CODE_STYLE.md`

Code style guides

- Подробные гайды от больших компаний (Google Style Guides)
- Охватывают форматирование, именование, комментарии, архитектуру
- Могут служить базой для своих внутренних правил
- Важно адаптировать под конкретный проект

Пример

`https://google.github.io/styleguide/pyguide.html`

Автоматическое соблюдение стиля

Проблема

- Большой документ с правилами люди не читают
- Люди забывают, спешат, ленятся

Решение

- Автоматическая проверка стиля и автоформатирование

Линтеры и Форматтеры

Линтеры

- Статический анализ кода
- Проверяют: стиль, форматирование и простые ошибки
- Могут быть строгими *(err)* или мягкими *(warn)*

Форматтеры

- Переформатируют код автоматически
- Отвечают только за внешний вид
- Правило: *«форматтер всегда прав»*.
  Без исключений и обсуждений

Главное отличие

- Линтер указывает на логические и стилистические ошибки (*«Здесь потенциальная проблема, исправь»*)
- Форматтер приводит всё к единому красивому виду (*«Мне все равно на логику, я просто расставлю пробелы»*)

Примеры линтеров

- **Python**: pylint
- **JavaScript/TypeScript**: ESLint
- **Java**: Checkstyle
- **Go**: golangci-lint
- **Rust**: clippy
- **C/C++**: clang-tidy
- **Docker**: hadolint
- **YAML**: yamllint
- **Terraform**: tflint
- **Markdown**: markdownlint

Примеры форматтеров

- **Python**: Black
- **JavaScript/TypeScript**: Prettier
- **Java**: google-java-format
- **Go**: gofmt
- **Rust**: rustfmt
- **C/C++**: clang-format
- **SQL**: sqlfluff
- **YAML**: prettier-yaml
- **Terraform**: terraform fmt
- **Markdown**: Prettier

Зачем использовать линтеры и форматтеры

- **Единообразный код**
  - Все пишут в одном стиле
- **Экономия времени на code review**
  - Не обсуждаем стиль, только логику кода
- **Меньше ошибок**
  - Линтеры ловят баги до review
- **Автоматизация в CI/CD**
  - Проверки запускаются на каждый commit/PR
  - Плохой код не попадёт в main

Где запускать линтеры и форматтеры

Локально

- В IDE: при сохранении файла
- Перед каждым коммитом

В пайплайне CI

- Каждый pull request автоматически прогоняет линтеры и тесты. Если что-то упало, **PR отклоняется до ревью**

Резюме блока

Проблема: «каждый пишет по-своему» решается

- **Документы**: README.md, CONTRIBUTING.md, CODE_STYLE.md
- **Инструменты**: линтеры, форматтеры
- **Процесс**: Автоматический reject некорректных PR

Как организовать параллельную работу нескольких людей над одной кодовой базой?

Git. Зачем он нужен?

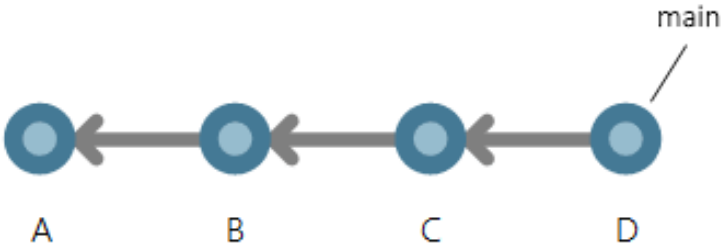
Определение

Git — распределённая система контроля версий.

Применение

- Отслеживает изменения в коде
- Возвращает к предыдущему состоянию проекта
- Позволяет безопасно экспериментировать
- Обеспечивает параллельную работу над проектом

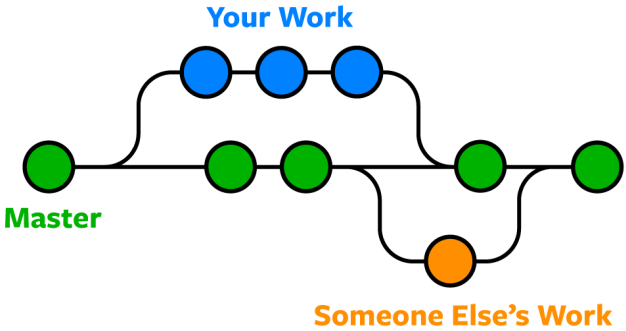
История изменений в Git



История — ориентированный граф коммитов

- Каждый коммит знает родителей
- Позволяет быстро просмотреть эволюцию файла
- Именованные коммиты -> Есть ответственные за изменения

Ветвление в Git



Ветки дешёвые

Свободное создание и быстрое переключение

Коллаборация Git + GitHub

Полный цикл командной разработки

1. Каждый разработчик работает локально
2. По готовности -> Push в удалённый репозиторий (GitHub, GitLab)
3. Создаёт Pull request: код-ревью, CI/CD

Результат

- Прозрачные, контроллируемые и обсуждаемые изменения

Аналоги Git: Subversion (SVN)

Централизованная система контроля версий

Единый центральный репозиторий. Клонируется проект, но не история

Плюсы

- Простое администрирование
- Единая история изменений - всё на сервере

Минусы

- Единая точка отказа
- Постоянная связь с сервером
- Тяжёлое ветвление/слияние

Аналоги Git: Mercurial

Распределённый Mercurial

Полный клон проекта и истории у каждого (как в Git)

Плюсы

- Простота команд
- Строгая модель истории

Минусы

- Меньше гибкости
- Меньше популярность в сообществе

Почему Git выиграл?

Киллер фичи Git

Распределённость + дешёвые ветвления + экосистема и сообщество

- GitHub, GitLab, IDE, CI/CD, pull requests
- Популярность в индустрии (>80% рынка)
- Альтернативы git охватывают свои небольшие ниши

Серверы Git-репозиториев

Три основных платформы

- **GitHub**
- **GitLab**
- **Bitbucket**

Главное отличие платформ — не в Git, а в:

- Веб-интерфейсе
- Процессах code review
- Интеграциях

Не путать

Git — протокол распределённого контроля версий
GitHub — веб платформа для хостинга Git-репозиториев.

GitHub

Преимущества

- Лидер рынка: огромное сообщество и экосистема
- Мощные Pull Requests с code review
- GitHub Actions для CI/CD

Когда использовать

- Open source проекты
- Большие распределённые команды

GitLab

Преимущества

- Self-hosted, open-source
- Встроенный CI/CD без внешних инструментов
- Безопасность в одной платформе

Когда использовать

- Собственный CI/CD
- Нужна независимость (self-hosted) и безопасность

Bitbucket

Atlassian

Преимущества

- Глубокая интеграция с Jira
- Branch permissions
- Поддержка Git + Mercurial

Когда использовать

- Atlassian экосистема (Jira, Trello, ...)

Как управлять ветками?

Стратегии ветвления

- Feature Branching
  - Git Flow
  - GitHub Flow
  - GitLab Flow
  - ...
- Continious Delivery
  - Trunk-based development

Стратегии управления ветками

«What if there was no branching. Everybody would be editing the live code, half-baked changes would bork the system, people would be stepping all over each other.
*And so **we give individuals the illusion of frozen time**, that they are the only ones changing the system and those changes can wait until they are fully baked before risking the system. But this is an illusion and eventually the **price for it comes due**.*
Who pays? When? How much?
That's what these patterns are discussing: alternatives for paying the piper.»

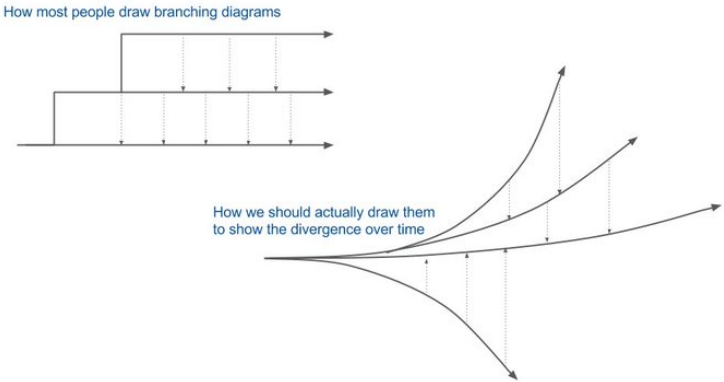
— ***Kent Beck***

Стратегии управления ветками

«An old joke says that if you fall off a tall building, the falling isn't going to hurt you, but the landing will.
So with source code: ***branching is easy, merging is harder****.»*

— ***Martin Fowler***

Отображение веток



Чем дольше ветка живёт - тем дальше она находится от main и тем сложнее merge

Термин «Healthy branch»

Ветка «здорова», если

- Программа в ней успешно собирается и работает корректно
- Запускаются и проходят автоматические проверки (тесты)

Как поддерживать «здоровье» ветки

- *Self Testing Code*: писать автотесты
- Запускать сборку и тесты на каждый коммит
- Код перестал запускаться - заморозить ветку до восстановления «здорового» состояния
  В неё больше нельзя делать обычные коммиты, только те изменения, которые чинят сборку

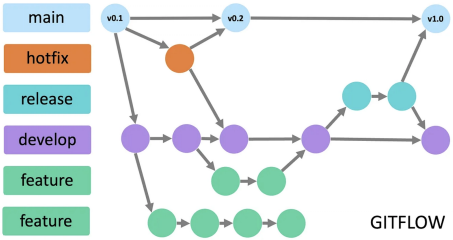
Стратегия «Feature branching»

Вся работа над фичёй ведётся в отдельной ветке, в main вливается только когда фича полностью закончена

Процесс

1. Создание ветки *feature/...*
2. Полная реализация фичи
3. Интеграция - merge, PR

Feature branching: Git Flow



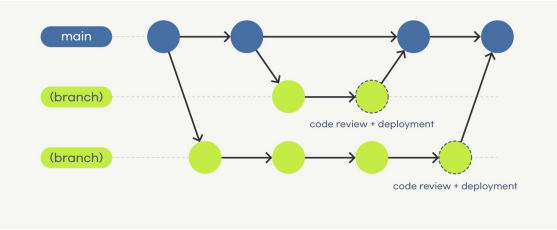
Описание:

- Несколько долгоживущих веток: `main`, `develop`
- Feature-ветки от `develop`
- Release и hotfix-ветки для управления версиями
- Строгое управление релизами

Когда использовать:

- Крупные проекты с длинным циклом разработки

Feature branching: GitHub Flow



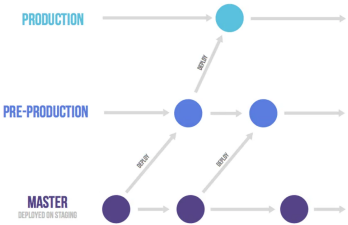
Описание:

- Одна основная ветка `main` (всегда production-ready)
- Короткоживущие feature-ветки
- Pull Request → Code Review → Merge

Когда использовать:

- Непрерывная доставка (CD)
- Частые деплои (несколько раз в день)
- Малые и средние команды

Feature branching: GitLab Flow



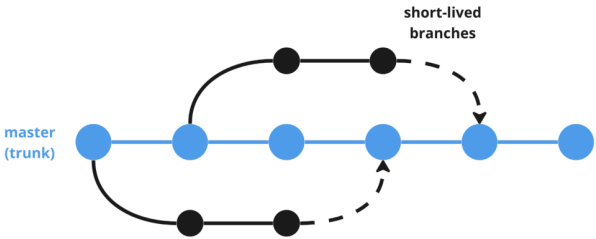
Описание:

- `main` - рабочая ветка
- Feature-ветки от `main`
- Environment-ветки: `production`, `pre-production`
- `main` → `pre-production` `(staging)` → `production`

Когда использовать:

- Несколько окружений (dev, stage, prod)
- Сложные CI/CD-процессы
- Необходимость стабильных релизов

Стратегия «Continious Integration»



Изменения сливаются в main сразу, как только появляется «здоровый» коммит: код собирается, тесты проходят

Процесс

1. Разработчик делает небольшое изменение (<1 часа работы)
2. Ветка «здорова» - изменения сливаются в main

Процесс CI на практике

1. Разработчик вносит небольшое изменение, локально запускает тесты
2. Перед отправкой обновляет свою ветку актуальной версией основной ветки (`pull`, `merge` или `rebase`) и решает конфликты
3. Повторно запускает тесты. Если всё хорошо — отправляет изменения в основную ветку
4. Сервер CI (Jenkins, GitHub Actions, GitLab CI) автоматически собирает проект и запускает тесты
5. При падении сборки или тестов восстановление работоспособности main — главный приоритет

CI и ветвление

Важно

- CI **не запрещает** использование веток для функций. Важно **когда** объединять изменения
- Короткоживущие ветки, которые часто вливаются в main, полностью совместимы с CI
- Многие команды применяют **trunk-based development** — разработку напрямую в общей основной ветке с очень небольшими и постепенными изменениями

Как работать с незавершёнными функциями?

Поскольку код попадает в основную ветку до финальной готовности функции, используют техники маскировки:

- Сначала реализуется **внутренняя логика**, затем **пользовательский интерфейс**
- **feature flags** — механизм для включения/отключения функциональности для пользователей без изменения кода. Незавершённый код может находиться в основной ветке, но будет скрыт

Преимущества непрерывной интеграции

Плюсы

- Конфликты между изменениями разных разработчиков небольшие и решаются быстро
- Частая интеграция и короткоживущие ветки обеспечивают высокую эффективность доставки ПО

Главный принцип CI

Поддерживать основную ветку всегда в рабочем состоянии

Чем чаще вы интегрируетесь — тем меньше боли при слиянии и выше стабильность продукта.

Что выбрать: Feature Branching или CI?

Feature Branching

- Цельные фичи в изоляции
- Удобный ревью целой логики
- Риск больших конфликтов

Continuous Integration

- Маленькие PR, частые мержи
- Быстрое обнаружение ошибок
- Требует строгого контроля `main`

CI — для скорости с хорошими тестами,
Feature Branching — для сложных задач.

“Джентльменские” правила

Atomic Commits

Определение

Один коммит решает **одну логическую задачу**:

- фича
- багфикс
- рефакторинг
- тест

Зачем?

- Упрощает откат изменений,
- облегчает ревью кода,
- помогает искать поломки в истории.

Atomic Commits: Пример

Плохо

- `commit "WIP: random changes"`

Хорошо

- `commit #1 "Add validation"`
- `commit #2 "Update tests"`

Rebase перед Push: Синхронизация с `main`

Смысл

- `git rebase origin/main` перед пушем
- избегайте merge-коммитов.

Зачем?

- Линейная, чистая история,
- без ненужных слияний.

Важно

- Rebase **только** в локальных ветках,
- не переписывайте общую историю.

Force Push

Force Push

- Не используйте `push -force`
- Исключение: только для локальных веток

Squash and merge: Объединение Коммитов

Сценарий

- Объединяйте хаотичные коммиты в **один чистый** перед слиянием.

Компромисс

- Чистая история,
- но потеря детализации.

Commit Message

Требования

- Понятный, в повелительном наклонении
- Пример: "Fix bug не "Fixed bug"

Работа с Ветками

Запреты

- Не делайте push в ветку, в которой идёт review
- Избегайте `force-push` на общие ветки

Анатомия Хорошего PR: Структура

Название и описание

- Чёткая проблема и решение
- Контекст
- Ссылка на issue

Размер

- 100 - 300 строк изменений

Анатомия Хорошего PR: Содержание

Требования

- 1 PR = 1 изменение
- Не смешивайте фичу и рефакторинг

Review: Тактика

Время

- Выделяйте отдельное время
- минимум 1–2 approval перед слиянием

Признаки Хорошего и Плохого Review

Хорошее ревью

- Аргументированные рекомендации
- Примеры кода

Плохое ревью

- "OK"

Культура и Этикет в PR

Вежливость

- Предлагайте ("Рассмотрите альтернативу")
- Не обвиняйте

Скорость

- Отвечайте в течение дня

Конфликты

- Не обсуждайте синхронно (мит, видеозвонки)
- Ведите хронологию переговоров

Источники информации и изображений

Источники информации

1. https://martinfowler.com/articles/branching-patterns.html

Источники изображений

1. https://www.nobledesktop.com/learn/git/git-branches
2. https://habr.com/ru/articles/508534/
3. https://productdock.com/trunk-based-development/
4. https://dev.to/truongpx396/git-flow-overview-and-common-git-commands-1odh

Photographs used under public domain or fair usage. No copyright infringement intended. If used accidently under wrong license please contact to be removed.

Программная инженерия

Лекция 2: Культура командной разработки

Денисов Артём Владимирович

Уральский Федеральный Университет

2026